A compression-free factorial study of role labels and call boundaries in code generation

Daniil Privezentsev[1*]

[1*]National Research University Higher School of Economics, Moscow, Russia.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Abstract

Role-labelled pipelines often change both the prompt and the number of model calls. We separate these choices in an uncompressed factorial experiment using GPT-5.4 mini with medium reasoning. Forty randomly selected development tasks, five conditions and two repeats yield 400 completed programs and 720 fresh Codex CLI turns. Complete exposed responses, forwarded history, token counters and native BigCodeBench reports are archived. Controls make 39 tasks evaluable. Direct solving passes 42/78 attempts at the lowest resource mean; single-call role prompting passes 39/78 and three-call role prompting 36/78. Role-minus-neutral quality differences are +6.41 and -3.85 percentage points in the two topologies; these are development estimates with task-level uncertainty. The matrix uses 3,703,821 tokens, valued at USD 6.61135 using published API prices, not a subscription charge. A real SWE-bench patch demonstration stops after a generation deadline: two observed patches fail and eight assignments remain missing. This auditable non-ceiling record weakens a general role-efficiency claim. A reserved-task study is being prepared; non-inferiority and native-agent superiority are not asserted.

Keywords: Code generation, role prompting, factorial design, token accounting, BigCodeBench, SWE-bench

1 Introduction: roles and call boundaries

Code-generation pipelines commonly describe stages as planning, implementation, and review. Two different interventions are often hidden inside that description. A role label can change the instructions given to one model response, while a call boundary

can split the work across separately sampled responses with intermediate text passed forward. If both interventions change together, an observed difference cannot be attributed to role wording, to repeated inference, or to the information exposed between calls. The present development study isolates this question at the protocol level: do role-labelled instructions behave differently from neutral stage instructions when the call topology is varied under the same task population and supplied context?

The estimand is deliberately narrower than a claim about independent agents or latent cognition. The study compares five implemented conditions: a direct one-call solver, a single-call response containing neutral stages, a single-call response containing role-labelled stages, a three-call neutral pipeline, and a three-call role-labelled pipeline. The single-call conditions place the three procedural stages in one model turn. The multi-call conditions expose each preceding response to the next stage. Thus call topology and intermediate information flow are part of the treatment definition. A result can support a statement about these frozen prompts and boundaries; it cannot establish that professional labels create independent internal agents.

2 Related work

The role-based framing has antecedents in programmatic prompting and internal dialogue. INoT organizes reasoning through explicit stages and instructions [1]. That prior work motivates a controlled comparison, but the existence of role names does not by itself identify a causal role effect. A useful comparison must separate role wording from the number of model calls and must preserve the information made available to each condition.

Budget and interaction structure are also distinct in related research. Work on single- and multi-agent reasoning under matched thinking-token budgets emphasizes that additional calls and additional computation are not interchangeable interventions [2]. The present study therefore records input, cached-input, and output counters and reports API-equivalent list-price valuation as a descriptive resource measure. It does not treat the subscription channel as an API invoice or claim that a token comparison is a complete cost-of-ownership comparison.

For software engineering, Agentless uses a structured localization–repair–validation workflow [3], whereas SWE-agent uses a tool-mediated repository interface [4]. Those systems address practical repository repair and include retrieval or interaction policies outside the fixed-context function tasks used here. BigCodeBench supplies diverse code-generation tasks with library calls and complex instructions [5]. It is suitable for testing instruction following and executable outputs, but its function-level distribution is not a substitute for repository repair. SWE-bench evaluates changes against repository tests [6, 7]; it remains a separate external boundary for this study.

3 Executed development methods

The executed extension uses GPT-5.4 mini with medium reasoning through Codex CLI 0.153.4. The runner uses the subscription channel and the frozen model alias; it does not claim an immutable weight snapshot. Each condition receives the same task

text and complete supplied context. In the multi-call conditions, every subsequent stage receives the full preceding exposed response. No input is compressed or selectively shortened for a particular arm. Tools, browsing, delegation, and test execution are disabled inside generation, and a strict JSONL trace is required for each fresh ephemeral thread.

The task population consists of 40 randomly ordered development tasks from BigCodeBench v0.1.4. The seeded development order and exact task assignment are recorded in the immutable manifest. Every task is retained, including a task whose reference control is unusable for correctness measurement. Two labelled repeats, IDs 2 and 3, are paired within task and condition. The labels are repeat identifiers rather than provider sampling seeds: the CLI does not expose a verified sampling seed and temperature is not controlled. Consequently, the repeats provide repeated observations under the same declared protocol without implying deterministic model replication.

The five conditions applied to 40 tasks and two repeat labels produce 400 assigned candidate programs. Single-call and direct conditions use one CLI turn per candidate; each three-stage condition uses three fresh turns. The full matrix therefore contains 720 fresh CLI turns. The ordered tasks are partitioned into four disjoint ten-task shards for each repeat wave. Four shards run in parallel, each with two runner workers, for a maximum of eight simultaneous CLI processes. Repeat wave 3 begins only after wave 2 has finished. Submission order within each shard is randomized by the frozen runner, while task membership and arm labels remain fixed by the manifest.

The runner imposes a 180-second deadline per CLI turn, a 65,536-byte pre-submission input guard, and no matched hard completion allowance across conditions. The absence of a matched allowance is a declared limitation: the design fixes the task information and non-treatment settings, but not actual completion length, reasoning usage, wall time, or total spending. No automatic retry, post-run repair of the archived final candidate, compression of exposed responses before forwarding, or favorable code-block selection is allowed. The implementation and review stages inside the declared treatment may revise code as instructed. If a shard fails, submitted events and observed usage are preserved, the already active wave is allowed to finish, and the next repeat wave is not started. A partial matrix remains partial, with unobserved assigned cells retained as missing.

Before model generation, gold and deliberately incorrect controls were executed for all 40 assigned tasks through the unchanged upstream BigCodeBench CLI, using the final isolated custom environment, original tests, network disabled, and the native "instruct" mode. The control gate records 39 tasks with a passing gold control and a failing incorrect control. One task has a gold failure and is therefore retained in the assignment but excluded from correctness measurement across all arms. The gate, image and package hashes, dataset and prepared-split hashes, source tree, commands, and raw control reports are archived. Passing a control establishes executability and negative discrimination; it does not certify that every benchmark assertion is semantically valid.

The independent unit is the task. Repeats are averaged within a task only for paired descriptive contrasts when both repeats and both compared conditions

are observable. The analysis reports evaluable-only rates together with assignment-denominator missingness bounds. Resource contrasts use complete paired task means; descriptive per-condition resource means retain their observed-candidate denominators when the matrix is partial. Task-cluster bootstrap intervals use 10,000 resamples and a fixed analysis seed. These are descriptive 95% bootstrap intervals without a confirmatory hypothesis test or a powered non-inferiority claim, and the 40 tasks do not become 400 independent observations merely because the matrix contains 400 candidates.

3.1 Treatment definitions and resource accounting

We use the following fixed abbreviations throughout: D, direct solving in one call; SN, three neutral stages in one call; SR, three role-labelled stages in one call; MN, neutral stages across three calls; MR, role-labelled stages across three calls. Neutral stages and named roles receive the same planning, implementation and review operations. The four structured conditions form a 2×2 topology-by-label factorial design; D is the simple external comparator.

The reviewer's topology-by-compression experiment would decompose the earlier package. The present question instead holds compression absent and varies role labels directly. It therefore isolates neither the historical compression effect nor topology independently of intermediate information flow. Removing a router and a small checker from the treatment also removes their untested effects from the contribution.

The CLI returns input I, cached-input C and output O counters for each completed turn [14]. Cached input is a subset of input; reasoning, when separately exposed as a counter, is a subset of output. Neither is added twice. We value each turn at the published GPT-5.4 mini standard API rates [12, 13]:

$$V = \frac{0.75(I - C) + 0.075C + 4.50O}{10^6} \quad \text{USD.} \tag{1}$$

The no-cache sensitivity is $V_0 = (0.75I + 4.50O)/10^6$. These are counterfactual list-price valuations of measured subscription-channel tokens, not API payments, subscription invoices or total cost of ownership. Research-assistant work, environment construction and evaluator execution are outside these generation totals. Caching and dispatch order can affect V; V_0 exposes that dependence. A provider-reported reasoning subset is not necessary to compute V, but its absence prevents a separate reasoning-token comparison.

4 Development results: complete execution and non-ceiling outcomes

All 400 assigned candidates and 720 CLI turns completed and passed the trace audit. Every saved final answer and token total reconciles to the original event stream; each multi-call input contains the exact complete prior exposed responses. The archive retains 3,703,821 observed tokens, valued at USD 6.6113451, or USD 7.7687595 in the no-cache sensitivity. No final-format extraction failed in this matrix. All 400 native

evaluator rows match the exact exported programs and task IDs. The failed reference control for task /1005 makes ten assigned outcomes unavailable, leaving 390 evaluable candidate attempts over 39 tasks.

Table 1 Development results: 80 assigned candidates per condition, two repeats on 40 tasks. Quality uses 78 evaluable attempts; resource means use all 80. Missingness ranges use the full assignment denominator and are not confidence intervals.

Arm	Passes	Rate (%)	Range (%)	Tokens	USD	No cache
D	42/78	53.85	52.50–55.00	4 500	0.00776	0.00939
SN	34/78	43.59	42.50–45.00	5 598	0.01251	0.01417
SR	39/78	50.00	48.75–51.25	5 189	0.01070	0.01234
MN	39/78	50.00	48.75–51.25	15 636	0.02624	0.03090
MR	36/78	46.15	45.00–47.50	15 375	0.02543	0.03030

The observed success rates range from 43.59% to 53.85%, so the earlier quality ceiling is absent. D has the largest observed success count (42/78) and the lowest resource mean. SR and MN each pass 39/78, MR passes 36/78, and SN passes 34/78. These are rates over separate attempts; no best-of-two selection is used. Test success is not automatically correctness relative to the natural-language instruction, given the contract issues discussed below.

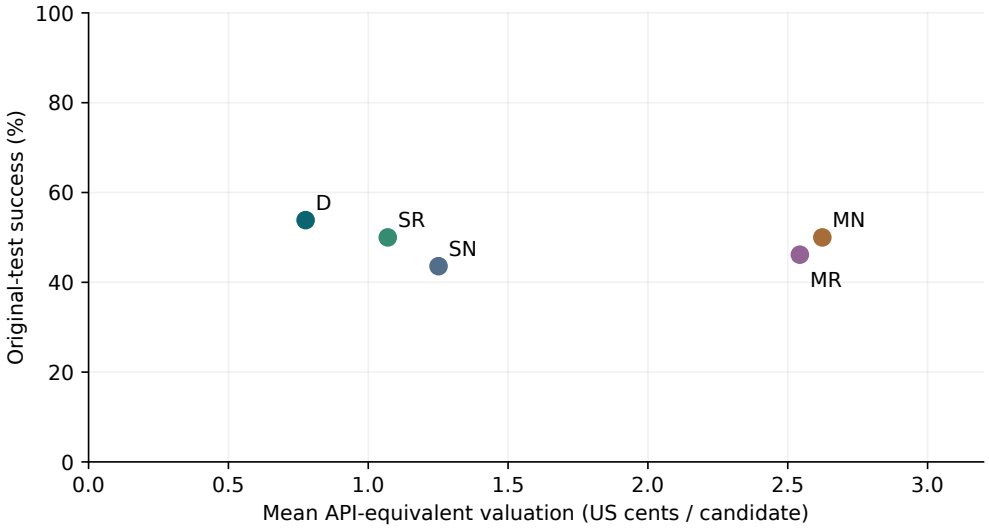

Figure 1 Observed development means. Labels use the five conditions defined above. Points summarize original-test success and API-equivalent valuation, not a confidence region or a benchmark-wide efficiency frontier. Quality and resource denominators differ only by the reference-ineligible task, whose usage remains counted.

Table 2 Paired development contrasts. Quality differences and descriptive 95% task-bootstrap intervals are percentage points over 39 complete task clusters; resource ratios use 40. The last two comparisons are exploratory. No confirmatory p-value or non-inferiority test is assigned.

Contrast	Quality difference [interval]	Token ratio	USD ratio
SR - SN	$+6.41$ $[+1.28, +11.54]$	0.927	0.855
MR - MN	-3.85 $[-11.54, +2.56]$	0.983	0.969
MN - SN	$+6.41$ $[-2.56, +15.38]$	2.793	2.097
MR - SR	-3.85 $[-8.97, +1.28]$	2.963	2.377
SN - D	-10.26 $[-19.23, -2.56]$	1.244	1.613
MR - D	-7.69 $[-17.95, +1.28]$	3.417	3.277

Role labels change quality differently across the two topologies: SR minus SN is $+6.41$ percentage points, while MR minus MN is -3.85 points. Their difference of differences is -10.26 points, with a descriptive task-bootstrap interval of $[-19.23, -1.28]$. This development interaction motivates a separate reserved-task check; it is not a confirmatory test result. The three-call conditions use 2.79 and 2.96 times the corresponding single-call token means. Their valuation ratios are 2.10 and 2.38 because cached input is cheaper than output. Thus fewer calls reduce measured resource use in this implementation, whereas the quality consequences depend on the prompt condition and have substantial uncertainty.

5 Repository-patch demonstration and an explicit incomplete run

A separate one-task SWE-bench Lite development demonstration uses `pvlib__pvlib-python-1606`. Deterministic BM25 retrieval from the exact base commit selects whole eligible Python files within 24,000 source bytes; nine files totaling 23,863 bytes enter the identical packet for all arms. Test files, hints and reference patches are excluded from retrieval. Files too large to fit are listed, not truncated. This is an explicitly selected packet, not the full repository, and omitted relevant files remain a retrieval limitation.

The unchanged pinned SWE-bench harness initially fails even with the gold patch because its official task image contains an incompatible NumPy version. A separate image changing NumPy to 1.26.4 passes all 11 gold tests; an applicable comment-only negative passes ten tests and fails the regression test. All initial failures and both image identities are retained. These controls validate the adjusted environment, not the unchanged release image. The controller runs offline, while the unchanged upstream child task container retains its default Docker network; no credentials or personal directories are mounted.

The fixed ten-candidate generation stops after one CLI call exceeds the 180-second deadline. Two complete patches remain: D applies but fails the target regression test (ten other tests pass); SN is rejected by native patch application. Eight assigned candidates remain missing. Both observed evaluations pass the input/patch/report audit

and have no identified infrastructure failure. Four CLI turns were submitted; three expose complete usage totaling 56,796 tokens and USD 0.1379895 in API-equivalent valuation, while one has unavailable usage. This is a partial development demonstration with no resolved instance, not a Lite benchmark score. It replaces the old surrogate `check()` route with real patch application and tests while retaining the failure to complete the planned matrix.

6 Limitations and external boundary

The executed design identifies the behavior of one requested model alias, one reasoning setting, one CLI version, five prompt conditions, and the selected 40-task development population. It does not identify a provider-stable model property, a universal role effect, or a causal effect independent of information flow. Because no provider sampling seed or temperature control is available, repeat labels should be interpreted as protocol-level repeated observations rather than exact stochastic replications. The absence of a matched hard output allowance leaves quality, completion length, and resource usage jointly informative but not budget-equated.

BigCodeBench test validity remains a separate concern. The independent instruction/test audit records prompt–test alignment issues as annotations, while original prompts, tests, and scores remain unchanged. A passing gold control is necessary for evaluability but cannot prove that every assertion measures the natural-language task. All assigned tasks remain in the denominator. Control or infrastructure failures produce missing outcomes; instruction/test disagreements remain annotations and do not erase their original scores. Task resampling handles repeated observations within a task but does not eliminate dependence between related benchmark problems.

The study is also bounded away from external repository-agent claims. It does not execute native INoT, Agentless, or SWE-agent systems, and it does not report an external INoT, Agentless, or SWE-agent benchmark result. Those systems differ in retrieval, tool use, repository state, patch application, and test interaction. A future comparison must freeze each native implementation, model and budget, collect complete trajectories, and use the corresponding official evaluator. In particular, a function-generation result cannot be relabelled as repository repair evidence, and a published external percentage cannot be inserted as a paired arm in this matrix.

The present study is consequently a bounded executed development experiment. It supplies an auditable comparison of role labels and call boundaries under full provided context, repeated exposed history, native BigCodeBench tests, and explicit resource accounting. It does not by itself justify a claim of latent multi-agent behavior, benchmark-wide superiority, non-inferiority, or external validity on contemporary repository engineering.

7 Interpretation and next reserved-task check

The study separates questions that were confounded in the earlier Hybrid-INoT package. Additional calls repeatedly transmit the input and exposed intermediate work; their measured resource use is higher here. Named roles show a positive quality contrast within one response but not across three calls. Direct solving has the strongest

observed development tradeoff. This supports retaining a simple baseline when evaluating internal-role methods; it does not prove that extra calls cannot help other tasks.

A separate allocation fixes 200 new tasks from the original seeded random permutation, excluding eight dataset-card examples incidentally exposed during license verification. No development task is reused. Five conditions and a new repeat label assign 1,000 candidates and 1,800 CLI turns. Controls precede generation; four paired quality contrasts have predeclared exact McNemar tests with Holm adjustment. This is not a powered two-point non-inferiority design. At this revision stage reserved model outcomes are unavailable and are not presented as executed results.

8 Conclusion

Complete responses and official tests replace a confounded efficiency claim with an executed comparison of role labels and call boundaries. On the development sample, direct solving combines the highest observed test success with the lowest token use and list-price valuation. Role effects differ across protocols, motivating reserved-task measurement. These auditable observations retain sampling, evaluator and provider-version limitations.

Data and code availability

The article repository [11] contains full generation and native evaluator archives, task/control/source hashes, original responses, extracted programs and per-task outcomes. The development manifest and control gate were fixed before dispatch. SHA-256 inventories link public artifacts. The earlier pilot, rejected attempt and historical records remain separate. No hidden provider reasoning or immutable weight snapshot is claimed; original source licenses accompany redistributed evidence.

Declarations

Funding and competing interests: no external funding or competing interest was reported in the preceding manuscript. **Research scope:** public software benchmarks; no human-participant study. **Responsibility:** AI assisted software, audits and revision; the author remains responsible for scientific claims. Software tests are not experimental model observations.

A Complete development outcomes

Table 3 Every development task and both repeats. Each pair of letters gives repeats 2 and 3 in order: P = pass, F = fail, U = unavailable. These are original test statuses after the reference eligibility gate; disputed tests have not been deleted.

ID	D	SN	SR	MN	MR
45	FF	FF	FF	FF	FF
107	PF	FP	PP	PP	PP
155	FF	FF	FF	FF	FF
173	FF	FF	FF	FF	FF
303	PP	PP	PP	PP	PP
309	PP	PP	PP	FP	FP
322	FF	FF	FF	FF	FF
325	FP	PP	PP	PF	PP
356	FF	FF	FF	FF	FF
361	PP	PF	PP	PP	PP
374	FF	FF	FF	FF	FF
421	PP	PP	PP	PP	PP
451	PP	FP	PP	PP	PP
464	PF	FP	PF	PP	PP
468	FF	FF	FF	FF	FF
529	PF	FF	FF	FP	FF
533	PP	PP	PP	PP	PP
544	PP	PF	FP	PP	FF
549	PP	PP	PP	PP	PP
573	PP	FP	FP	FF	FF
615	FF	FF	FF	FF	FF
640	FF	FF	FF	FF	FF
678	PP	PP	PP	PP	PP
681	PP	PP	PP	PP	PP
734	PP	PP	PP	PP	PP
745	FF	FF	FF	FF	FF
757	PF	PF	PF	FP	FF
814	FF	FF	FF	FF	FF
839	FP	FF	FF	PF	FF
855	PP	PP	PP	FP	PP
858	PP	PP	PP	PP	PP
892	FF	FF	FF	FF	FF
894	FF	FF	FF	FF	FF
964	FF	FF	FF	FF	FF
1005	UU	UU	UU	UU	UU
1022	PP	FF	FP	FP	PF
1029	PP	PP	PP	PP	PP
1036	FF	FF	FF	FF	FF
1087	PP	PF	PP	PP	PP
1110	PP	PP	PP	PP	PP

B Historical evidence and calibration

The earlier 240 Flash-Lite records cover 20 tasks, four target lengths, three architectures and one seed, not 240 independent tasks. Saved quality labels are all one and full solutions are missing: arithmetic is checkable, correctness cannot be replayed. B3 has an extra rerun in 48/80 records; its final labels are not single-attempt pass@1. Its comparison with B2 mixes roles, calls, reruns and context selection. Relative to the simple B0 solver it costs 10.3–85.0% more at the three shorter contexts; the longest-context saving coincides with input selection. These records motivate the replacement but do not enter its estimates.

The preceding eight-task pilot generated 40 candidates in 72 turns at USD 0.7417401 API-equivalent valuation. On seven evaluable tasks, direct solving passed 4/7, single-call roles 3/7 and three-call roles 2/7. The extension uses fresh responses. An earlier CLI 0.144.1 attempt was rejected after an unexpected planning tool appeared; all 21 submitted turns and known USD 0.2493285 valuation remain archived. Rejected responses are never spliced into the valid matrix.

References

[1] H. Sun and S. Zeng. Introspection of Thought Helps AI Agents. arXiv:2507.08664v1, 2025. https://arxiv.org/abs/2507.08664v1.

[2] D. Tran and D. Kiela. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460v2, 2026. https://arxiv.org/abs/2604.02460v2.

[3] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024. https://arxiv.org/abs/2407.01489.

[4] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS, 2024. https://arxiv.org/abs/2405.15793.

[5] T. Y. Zhuo et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. ICLR, 2025. https://arxiv.org/abs/2406.15877.

[6] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. https://arxiv.org/abs/2310.06770.

[7] SWE-bench maintainers. Evaluation Guide. Accessed 8 September 2026. https://www.swebench.com/SWE-bench/guides/evaluation/.

[8] OpenAI. Why SWE-bench Verified no longer measures frontier coding capabilities. 2026. https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/.

[9] OpenRouter. Models API and provider usage metadata. Snapshot accessed 8 September 2026. https://openrouter.ai/api/v1/models.

[10] D. Privezentsev. INoT_Research. Historical implementation, commit `010211ee5775185e8330ab6cde06e912c2adcb9e`. https://github.com/daniil-novel/INoT_Research.

[11] D. Privezentsev. Article-INoT: manuscript, historical artifacts and compression-free revision protocol. 2026. https://github.com/daniil-novel/article-INoT.

[12] OpenAI. GPT-5.4 mini model documentation. Accessed 8 September 2026. https://developers.openai.com/api/docs/models/gpt-5.4-mini.

[13] OpenAI. API pricing, standard text-token rates. Accessed 8 September 2026. https://developers.openai.com/api/docs/pricing.

[14] OpenAI. Codex non-interactive mode and JSONL event stream. Accessed 8 September 2026. https://developers.openai.com/codex/noninteractive.

[15] OpenAI. Codex CLI 0.152.0 release notes: planning tool configuration. 1 September 2026. https://github.com/openai/codex/releases/tag/rust-v0.152.0.